



hexagone

École Supérieure d'Informatique

Fichiers et données

4PYTH – Langage Python

Chris CHEVALIER



hexagone

École Supérieure d'Informatique

Sommaire

- #1** Lire et écrire des fichiers
- #2** Octets et empreintes
- #3** JSON et sauvegarde d'objets



#1

Lire et écrire des fichiers

Fichiers et données



Ouvrir un fichier

```
notes.py

with open("notes.txt", "w", encoding="utf-8") as f:
    f.write("Audit du serveur srv-web\n")

with open("notes.txt", encoding="utf-8") as f:
    content = f.read()
print(content)    # Audit du serveur srv-web
```

Mode	Effet
"r"	lecture (par défaut)
"w"	écriture : écrase le fichier
"a"	ajout à la fin

- `with` **ferme le fichier** automatiquement, même en cas d'erreur

Piège : précisez toujours `encoding="utf-8"` . Sans lui, Windows utilise un autre encodage et les accents deviennent illisibles.



Lire ligne par ligne

```
auth.log
Oct  9 10:02:11 srv-web sshd[2210]: Failed password for root from 203.0.113.45 port 51122 ssh2
Oct  9 10:15:44 srv-web sshd[2303]: Failed password for root from 198.51.100.23 port 40031 ssh2
Oct  9 11:31:02 srv-web sshd[2410]: Accepted password for carol from 192.0.2.10 port 42230 ssh2
```

```
analyse.py
failures = {}
with open("auth.log", encoding="utf-8") as log:
    for line in log:
        if "Failed password" in line:
            words = line.split()
            ip = words[words.index("from") + 1]
            failures[ip] = failures.get(ip, 0) + 1
print(failures)
# {'203.0.113.45': 1, '198.51.100.23': 1}
```

- `for line in log` lit **une ligne à la fois**
- La mémoire reste stable, même pour un journal de plusieurs Go
- C'est votre défi 2... avec un vrai fichier



Écrire et ajouter

```
journal.py

from datetime import datetime

def log_event(message):
    now = datetime.now().isoformat(timespec="seconds")
    with open("actions.log", "a", encoding="utf-8") as f:
        f.write(f"{now} {message}\n")

log_event("Ajout du membre Jagger Fount")
log_event("Retrait du membre Hugo Crast")
```

```
actions.log

2026-10-09T10:42:05 Ajout du membre Jagger Fount
2026-10-09T10:42:05 Retrait du membre Hugo Crast
```

- Le mode `"a"` **ajoute** : relancer le programme allonge le journal
- `write()` n'ajoute pas de retour à la ligne : `\n` est à votre charge



Les erreurs de fichier

```
lecture.py

try:
    with open("rapport.txt", encoding="utf-8") as f:
        print(f.read())
except FileNotFoundError:
    print("Fichier introuvable")
except PermissionError:
    print("Accès refusé : droits insuffisants")
```

```
chemins.py

from pathlib import Path

path = Path("auth.log")
print(path.exists())    # True
print(path.suffix)      # .log
```

- Un fichier peut manquer ou être protégé : le programme ne doit pas planter
- `pathlib` manipule les chemins sous Windows, macOS et Linux



#2

Octets et empreintes

Fichiers et données



Texte ou octets ?

```
encodage.py

text = "Clé"
data = text.encode("utf-8")
print(data)          # b'Cl\xc3\xa9'
print(len(text))     # 3 caractères
print(len(data))     # 4 octets
print(data.decode("utf-8")) # Clé
```

- `str` : du **texte**, pour les humains
- `bytes` : des **octets**, pour les machines (fichiers, réseau, cryptographie)
- `encode()` : texte → octets ;
`decode()` : octets → texte
- En UTF-8, `é` occupe **2 octets**



L'hexadécimal

```
hexa.py
data = "Hi!".encode()
print(data.hex())           # 486921
print(bytes.fromhex("486921")) # b'Hi!'

print(int("ff", 16))        # 255
print(hex(443))             # 0x1bb
```

- Un octet s'écrit avec **2 chiffres hexadécimaux** : 48 = H
- C'est la notation des empreintes, des adresses mémoire et des éditeurs hexadécimaux
- `int(texte, 16)` convertit depuis la base 16

Vous la retrouverez partout en **4FSIC** Forensic et en **5REVE** Reverse engineering.



Lire un fichier binaire

```
signature.py

SIGNATURES = {
    b"%PDF": "PDF",
    b"\x89PNG": "PNG",
    b"PK": "ZIP (ou docx, xlsx...)",
}

def detect_type(path):
    with open(path, "rb") as f:
        header = f.read(8)
        for magic, kind in SIGNATURES.items():
            if header.startswith(magic):
                return kind
        return "inconnu"

print(detect_type("facture.pdf")) # PDF
print(detect_type("photo.jpg")) # PNG !
```

- `"rb"` : lecture **binaire**, on obtient des `bytes`
- Les premiers octets d'un fichier, les **magic bytes**, révèlent son vrai format
- `photo.jpg` est en réalité un PNG

On reconnaît un fichier à son **contenu**, pas à son extension : un réflexe de base du forensic.



Empreinte SHA-256

```
empreinte.py

import hashlib

print(hashlib.sha256(b"Fondation").hexdigest())
# 9dff5c74a4ea8611...eef425fd
print(hashlib.sha256(b"fondation").hexdigest())
# 7e1f6f4f4672b3fa...3a988cbc

def file_sha256(path):
    digest = hashlib.sha256()
    with open(path, "rb") as f:
        while True:
            chunk = f.read(65536)  # blocs de 64 Ko
            if not chunk:
                break
            digest.update(chunk)
    return digest.hexdigest()
```

- Une **empreinte** (hash) résume n'importe quelles données en 64 caractères hexadécimaux
- Une seule lettre change : l'empreinte change **entièrement**
- Comparer deux empreintes permet de vérifier l'**intégrité** d'un fichier

SHA-256 seul ne suffit pas pour stocker des mots de passe. Vous verrez pourquoi au TP4.



Qu'affiche ce programme ?

```
quiz.py  
  
word = "café"  
data = word.encode()  
print(len(word), len(data))  
print(data[-2:].hex())  
print(int("10", 16))
```

4 5 : 4 caractères mais 5 octets, car é en occupe 2

c3a9 : les 2 derniers octets, ceux du é

16 : 10 en base 16 vaut 16



#3

JSON et sauvegarde d'objets

Fichiers et données



Le format JSON

- **JSON** est un format texte de données structurées, lisible par les humains
- C'est le format des API web, des fichiers de configuration... et des sauvegardes du TP3

JSON	Python
objet {}	dict
tableau []	list
chaîne, nombre	str , int , float
true / false / null	True / False / None

```
config.json
{
  "server": "srv-web",
  "ports": [22, 443],
  "mfa": true,
  "owner": "Équipe SOC"
}
```



Écrire et lire du JSON

```
config.py

import json

config = {"server": "srv-web", "ports": [22, 443],
          "mfa": True, "owner": "Équipe SOC"}

with open("config.json", "w", encoding="utf-8") as f:
    json.dump(config, f, indent=2, ensure_ascii=False)

with open("config.json", encoding="utf-8") as f:
    loaded = json.load(f)
    print(loaded["ports"])           # [22, 443]

text = json.dumps(config)           # dict → str
print(json.loads(text)["mfa"])      # True
```

- `dump` / `load` : vers ou depuis un **fichier**
- `dumps` / `loads` : vers ou depuis une **chaîne** (le « s » de *string*)
- `indent=2` : un fichier lisible
- `ensure_ascii=False` : les accents restent lisibles



Sauvegarder des objets

```
user.py

class User:
    def __init__(self, username, role):
        self.username = username
        self.role = role

    def to_dict(self):
        return {"username": self.username, "role": self.role}

def user_from_dict(data):
    return User(data["username"], data["role"])
```

```
main.py

json.dumps(users)
# TypeError: Object of type User
# is not JSON serializable

data = []
for user in users:
    data.append(user.to_dict())
json.dump(data, f, indent=2)
```

- JSON ne connaît pas vos classes : on convertit chaque objet en **dictionnaire**
- Au chargement, `user_from_dict()` refait l'objet
- C'est **la** difficulté du TP3 : chaque classe aura son `to_dict()`



Charger sans planter

```
sauvegarde.py

def load_users(path):
    try:
        with open(path, encoding="utf-8") as f:
            data = json.load(f)
    except FileNotFoundError:
        return [] # premier lancement
    except json.JSONDecodeError:
        print("Sauvegarde corrompue")
        return []
    users = []
    for item in data:
        users.append(user_from_dict(item))
    return users

print(len(load_users("absent.json"))) # 0
print(len(load_users("users.json"))) # 2
```

- Au **premier lancement**, la sauvegarde n'existe pas encore : ce n'est pas une erreur
- Un fichier tronqué ou modifié à la main lève `json.JSONDecodeError`
- Dans les deux cas, le programme démarre avec des données vides



Côté sécurité

```
user.py

ROLES = ["analyste", "auditeur", "admin"]

def user_from_dict(data):
    if data.get("role") not in ROLES:
        raise ValueError(f"rôle inconnu : {data.get('role')}")
    return User(data["username"], data["role"])

user_from_dict({"username": "eve", "role": "root"})
# ValueError: rôle inconnu : root
```

- ❖ **Jamais** de `pickle` ni d'`eval()` sur des données non fiables : ils peuvent **exécuter du code** caché dans le fichier
- ❖ `json` ne fait que lire des données : il est sûr
- ❖ Mais un fichier JSON se modifie à la main : les données chargées **se valident**, par liste blanche comme au TP1

Au TP4, une **empreinte SHA-256** de la sauvegarde permettra de détecter toute modification du fichier.



Récapitulatif

Notion	Où elle sert
Classes, héritage, polymorphisme, composition	TP2 : Person , Operator , Mentalist , Spaceship , Fleet
Lecture ligne par ligne, <code>bytes</code> , hexadécimal, <code>hashlib</code>	Défis de cet après-midi
JSON, <code>to_dict()</code> , chargement robuste	TP3 : sauvegarde et chargement de la flotte
Journal horodaté, empreinte SHA-256	TP4 : sécurisation du projet, le 5 novembre



Cet après-midi

Horaire	Programme
14h00 – 15h15	Défis du jour 2 : fichiers, octets et empreintes
15h15 – 16h45	TP2 – Fondation : ajout des classes. Le TP3 est à avancer en autonomie
16h45 – 17h00	Kahoot : 10 questions sur la journée

Les défis sont **notés en séance** :
appelez-moi pour me montrer chaque défi
réussi.

Parcours confirmé : défis avancés en
parallèle, puis bonus des TP2 et TP3.



Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com